Lab 01:
Write a program to find the largest and smallest element in an array.

```
#include <stdio.h>
int main()
{
int i,a[10],min,max;
printf("Enter 10 integers: ");

for(i=0;i<10;i++)
{
scanf("%d",&a[i]);
}
min=a[0];
max=a[0];
for(i=1;i<10;i++)
{
if(a[i]<min)
min=a[i];
if(a[i]>max)
max=a[i];
}
printf("The largest number is: %d\n",max);
printf("The smallest number is: %d\n",min);
}
```

```
miruthi@Miruthis-MacBook-Air Desktop % gcc q1.c -o q1
miruthi@Miruthis-MacBook-Air Desktop % ./q1
Enter 10 integers: 23 21 37 65 38 40 87 35 64 24
The largest number is: 87
The smallest number is: 21
```

Lab 02:
Write a program to find the factorial of a number using recursion.
Write a program to find the nth Fibonacci number using recursion

```
#include <stdio.h>
int fact(int n)
{
if(n<=1)
{
return 1;
}
else
{
return n*fact(n-1);
}}
int main()
{
int a;
printf("Enter a number: ");
scanf("%d",&a);
printf("The factorial is: %d",fact(a));
}
```

```
miruthi@Miruthis-MacBook-Air Desktop % gcc q2.c -o q2
miruthi@Miruthis-MacBook-Air Desktop % ./q2
Enter a number: 5
The factorial is: 120%
```

Lab 04:
Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next.
Implement the following operations:
1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list.
2. .insert_at_end(data): Insert a new node with the given data at the end of the list.
3. delete_node(data): Delete the first node in the list that contains the given data
4. traverse (): Traverse the list and print the data of each node

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void beginning(int data) {
    struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));

    newnode->data = data;
    newnode->next = head;
    head = newnode;
}

void end(int data) {
    struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = data;
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
        return;
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newnode;
}

void delnode(int data) {
    struct Node *temp = head, *prev = NULL;

    if (temp != NULL && temp->data == data) {
        head = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != data) {
        prev = temp;
        temp = temp->next;
    }
```

```
}

void traverse() {
    struct Node* temp = head;

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    beginning(10);
    beginning(5);
    end(20);
    end(30);

    printf("Linked List:\n");
    traverse();

    delnode(20);

    printf("After deletion:\n");
    traverse();

    return 0;
}
```

```
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab4.c -o lab4
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab4
Linked List:
5 10 20 30 NULL
After deletion:
5 10 30 NULL
miruthi@Miruthis-MacBook-Air DSA Lab %
```

Lab05:
Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Stack to represent the stack itself. Implement the following operations:
1. push(data): Push a new node with the given data onto the stack.
2. pop(): Remove and return the top node from the stack. If the stack is empty, return None.
3. peek(): Return the data of the top node without removing it. If the stack is empty, return None
4. empty(): Return True if the stack is empty, otherwise return False

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;


void push(int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    if (new_node == NULL) {
        printf("Stack Overflow\n");
        return;
    }

    new_node->data = data;
    new_node->next = top;
    top = new_node;
}


int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }

    struct Node* temp = top;
    int popped = temp->data;
    top = top->next;
    free(temp);

    return popped;
}


int peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return -1;
    }
    return top->data;
}
```

```c
}


int main() {
    push(10);
    push(20);
    push(30);

    printf("Top element: %d\n", peek());

    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());

    printf("Is stack empty? %s\n", empty() ? "Yes" : "No");

    printf("Popped: %d\n", pop());

    printf("Is stack empty? %s\n", empty() ? "Yes" : "No");

    return 0;
}
```

```
miruthi@Miruthis-MacBook-Air DSA Lab % touch lab5.c
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab5.c -o lab5
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab5
Top element: 30
Popped: 30
Popped: 20
Is stack empty? No
Popped: 10
Is stack empty? Yes
miruthi@Miruthis-MacBook-Air DSA Lab %
```

Lab06:
Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Queue to represent the queue itself. Implement the following operations:
1. enqueue(data): Add a new node with the given data to the end of the queue.
2. front(): Return the data of the front node without removing it. If the queue is empty, return None.
3. is_empty(): Return True if the queue is empty, otherwise return False

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
struct Node *front = NULL, *rear = NULL;


void enqueue(int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    if (new_node == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    new_node->data = data;
    new_node->next = NULL;


    if (rear == NULL) {
        front = rear = new_node;
        return;
    }

    rear->next = new_node;
    rear = new_node;
}


int get_front() {
    if (front == NULL) {
        return -1;  // indicates empty queue
    }
    return front->data;
}


int is_empty() {
    return front == NULL;
}


void traverse() {
    struct Node* temp = front;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
```

```c
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    printf("Queue elements: ");
    traverse();

    printf("Front element: %d\n", get_front());

    printf("Is queue empty? %s\n", is_empty() ? "Yes" : "No");

    return 0;
}
```

Output:

```
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab6.c -o lab6
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab6
Queue elements: 10 -> 20 -> 30 -> NULL
Front element: 10
Is queue empty? No
miruthi@Miruthis-MacBook-Air DSA Lab %
```

Lab 07:

Write a program to create a class/structure DoublyNode to represent a node in a doubly linked list. Each node should have three attributes: data, next, and prev. Then, create a class/structure DoublyLinkedList to represent the linked list itself. Implement the following operations:

1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list.
2. insert_at_end(data): Insert a new node with the given data at the end of the list.
3. traverse_forward (): Traverse the list forward and print the data of each node.

```c
#include <stdio.h>
#include <stdlib.h>

struct DoublyNode {
    int data;
    struct DoublyNode* next;
    struct DoublyNode* prev;
};


struct DoublyNode* head = NULL;

void beginning(int data) {
    struct DoublyNode* newnode = (struct DoublyNode*)malloc(sizeof(struct DoublyNode));

    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = head;

    if (head != NULL) {
        head->prev = newnode;
    }

    head = newnode;
}


void end(int data) {
    struct DoublyNode* newnode = (struct DoublyNode*)malloc(sizeof(struct DoublyNode));
    newnode->data = data;
    newnode->next = NULL;


    if (head == NULL) {
        newnode->prev = NULL;
        head = newnode;
        return;
    }

    struct DoublyNode* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newnode;
    newnode->prev = temp;
}
```

```c
}


int main() {
    beginning(10);
    beginning(5);
    end(20);
    end(30);

    printf("Doubly Linked List (Forward):\n");
    traverseforward();

    return 0;
}
```

Output:

```
miruthi@Miruthis-MacBook-Air DSA Lab % touch lab7.c
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab7.c -o lab7
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab7
Doubly Linked List (Forward):
5 <-> 10 <-> 20 <-> 30 <-> NULL
miruthi@Miruthis-MacBook-Air DSA Lab %
```

Lab 08:
Write a program to a class/structure CircularNode to represent a node in a circular linked list. Each node should have two attributes: data and next. Then, create a class/structure CircularLinkedList to represent the linked list itself. Implement the following operations:
1.insert_at_beginning(data): Insert a new node with the given data at the beginning of the list.
2. traverse(): Traverse the list and print the data of each node

```
#include <stdio.h>
#include <stdlib.h>


struct CircularNode {
    int data;
    struct CircularNode* next;
};


struct CircularNode* head = NULL;


void beginning(int data) {
    struct CircularNode* new_node = (struct CircularNode*)malloc(sizeof(struct CircularNode));
    new_node->data = data;


    if (head == NULL) {
        new_node->next = new_node;
        head = new_node;
        return;
    }

    struct CircularNode* temp = head;


    while (temp->next != head) {
        temp = temp->next;
    }

    new_node->next = head;
    temp->next = new_node;
    head = new_node;
}

void traverse() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct CircularNode* temp = head;

    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
```

```
    printf("(back to head)\n");
}


int main() {
    beginning(10);
    beginning(5);
    beginning(2);

    printf("Circular Linked List:\n");
    traverse();

    return 0;
}
```

Output:

```
miruthi@Miruthis-MacBook-Air DSA Lab % touch lab8.c
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab8.c -o lab8
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab8
Circular Linked List:
2 -> 5 -> 10 -> (back to head)
miruthi@Miruthis-MacBook-Air DSA Lab %
```

Lab 09

Write a program to create a class/structure TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself. Implement the following operations:

1. insert(data): Insert a new node with the given data into the binary tree (assume a simple insertion without balancing).
2. pre_order_traversal(): Perform a pre-order traversal of the tree and print the data of each node

```c
#include <stdio.h>
#include <stdlib.h>


struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};


struct TreeNode* root = NULL;

struct TreeNode* createNode(int data) {
    struct TreeNode* new_node = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}


struct TreeNode* insert(struct TreeNode* root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    } else {
        root->right = insert(root->right, data);
    }

    return root;
}

void pre_order_traversal(struct TreeNode* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        pre_order_traversal(root->left);
        pre_order_traversal(root->right);
    }
}


int main() {
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
```

```c
}

int main() {
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    printf("Pre-order Traversal:\n");
    pre_order_traversal(root);

    return 0;
}
```

Output:

```
miruthi@Miruthis-MacBook-Air DSA Lab % touch lab9.c
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab9.c -o lab9
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab9
Pre-order Traversal:
50 30 20 40 70 60 80 %
miruthi@Miruthis-MacBook-Air DSA Lab %
```

Lab10:

Write a program to create a class TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself. include a method/function find_lca(node1, node2) that takes two node values as input and returns the value of their lowest common ancestor. Assume all values in the tree are unique.

```c
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

struct TreeNode* createNode(int data) {
    struct TreeNode* new_node = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}


struct TreeNode* insert(struct TreeNode* root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

struct TreeNode* find_lca(struct TreeNode* root, int n1, int n2) {
    if (root == NULL)
        return NULL;

    if (n1 < root->data && n2 < root->data)
        return find_lca(root->left, n1, n2);

    if (n1 > root->data && n2 > root->data)
        return find_lca(root->right, n1, n2);

    return root;
}

int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
```

```c
int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    int n1 = 20, n2 = 40;

    struct TreeNode* lca = find_lca(root, n1, n2);

    if (lca != NULL)
        printf("LCA of %d and %d is: %d\n", n1, n2, lca->data);
    else
        printf("LCA not found\n");

    return 0;
}
```

Output:

```
miruthi@Miruthis-MacBook-Air DSA Lab % touch lab10.c
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab10.c -o lab10
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab10
LCA of 20 and 40 is: 30
miruthi@Miruthis-MacBook-Air DSA Lab %
```

```c
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};


struct TreeNode* createNode(int data) {
    struct TreeNode* new_node = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}

struct TreeNode* insert(struct TreeNode* root, int data) {
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

struct TreeNode* findNode(struct TreeNode* root, int key) {
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return findNode(root->left, key);
    else
        return findNode(root->right, key);
}


void find_grandchildren(struct TreeNode* root, int key) {
    struct TreeNode* node = findNode(root, key);

    if (node == NULL) {
        printf("Node not found\n");
        return;
    }
```

Lab 11

Write a program to create a class TreeNode to represent a node in a binary tree. Each node should have three attributes: data, left, and right. Then, create a class/structure BinaryTree to represent the binary tree itself, to include a method/function find_grandchildren(node) that takes a node value as input and returns a list of values of all the grandchildren of the given node.

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};


struct TreeNode* createNode(int data) {
    struct TreeNode* new_node = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}

struct TreeNode* insert(struct TreeNode* root, int data) {
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

struct TreeNode* findNode(struct TreeNode* root, int key) {
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return findNode(root->left, key);
    else
        return findNode(root->right, key);
}


void find_grandchildren(struct TreeNode* root, int key) {
    struct TreeNode* node = findNode(root, key);

    if (node == NULL) {
        printf("Node not found\n");
        return;
    }
```

```
    if (node->left != NULL) {
        if (node->left->left != NULL)
            printf("%d ", node->left->left->data);
        if (node->left->right != NULL)
            printf("%d ", node->left->right->data);
    }


    if (node->right != NULL) {
        if (node->right->left != NULL)
            printf("%d ", node->right->left->data);
        if (node->right->right != NULL)
            printf("%d ", node->right->right->data);
    }

    printf("\n");
}

// Main function
int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    find_grandchildren(root, 50);

    return 0;
}
```

Output:

```
miruthi@Miruthis-MacBook-Air DSA Lab % touch lab11.c
miruthi@Miruthis-MacBook-Air DSA Lab % gcc lab11.c -o lab11
miruthi@Miruthis-MacBook-Air DSA Lab % ./lab11
Grandchildren of 50: 20 40 60 80
miruthi@Miruthis-MacBook-Air DSA Lab %
```